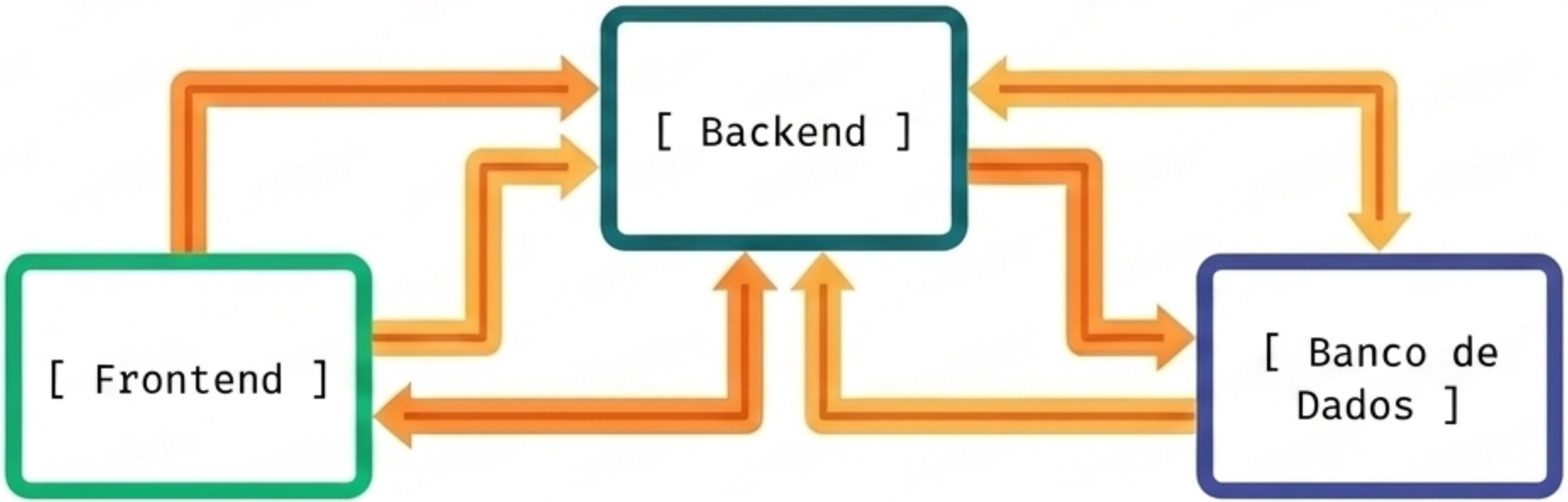
A Anatomia de uma Aplicação Web Moderna

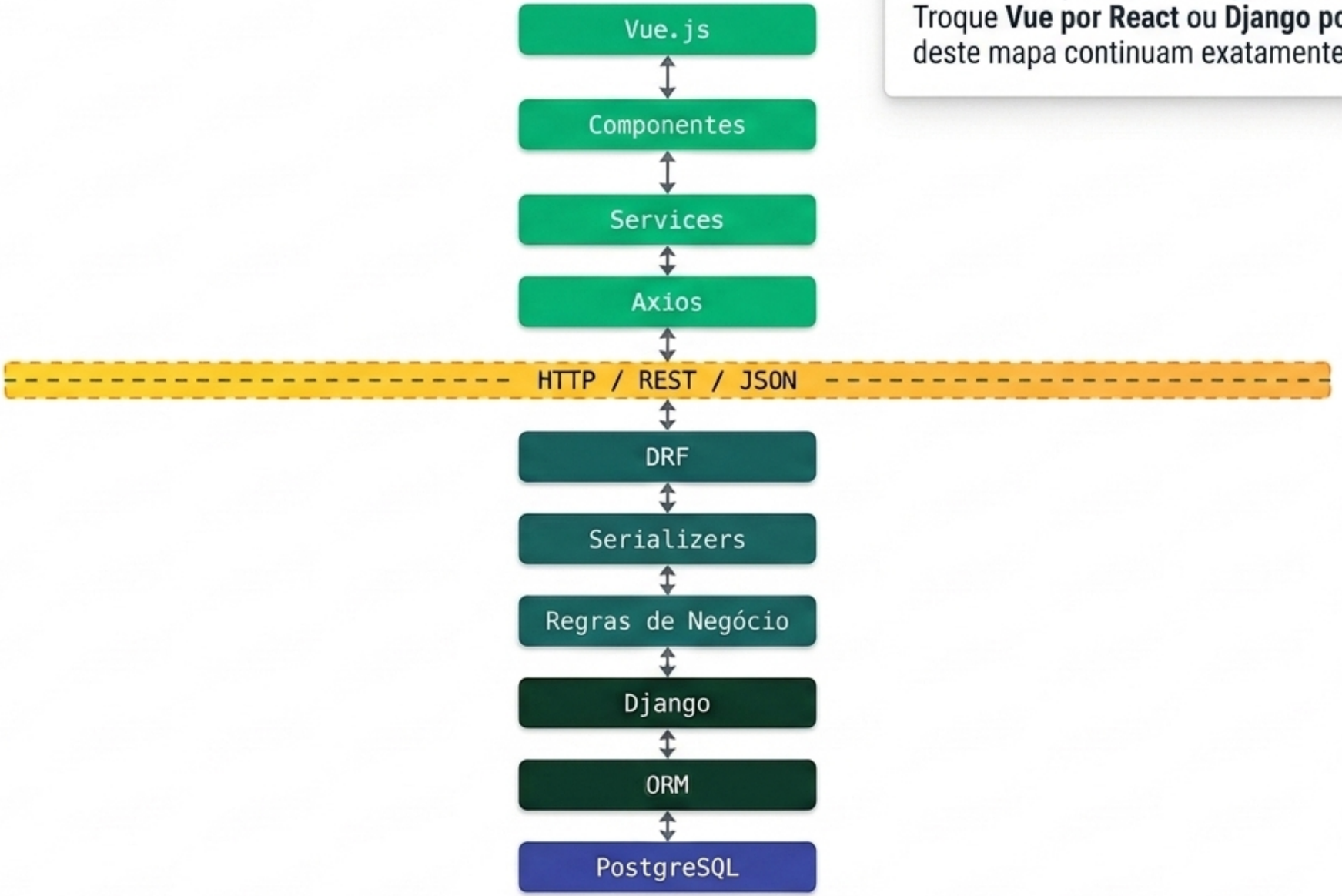
Disciplina: Desenvolvimento para Dispositivos Móveis | Prof. Eduardo da Silva

Do banco de dados à interface do usuário: um mapa mental para entender a integração dos sistemas web.



Gemini Notebook

A stack é específica. A **arquitetura é universal**. Troque **Vue por React** ou **Django por Node**, e as regras deste mapa continuam exatamente as mesmas.

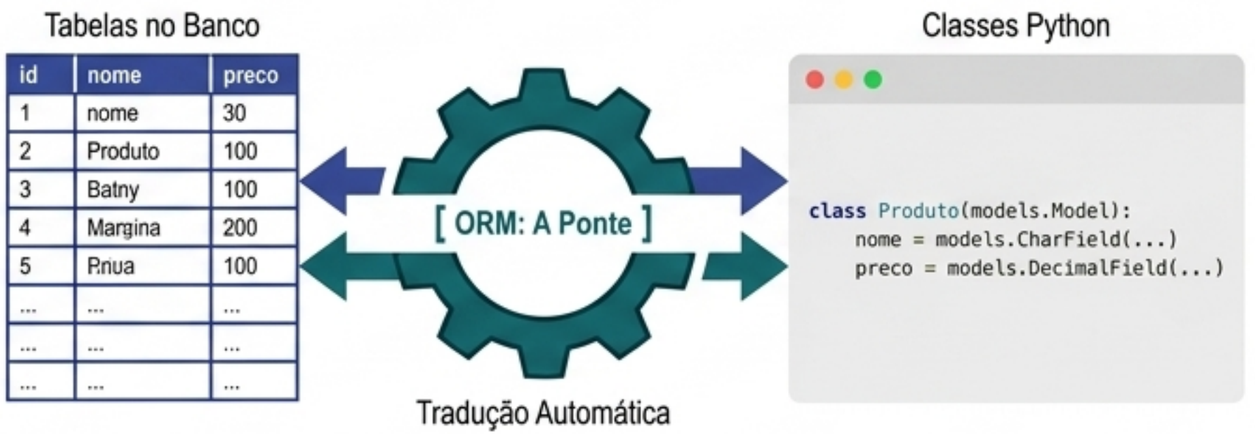


O ecossistema completo funciona através de uma estrita separação de responsabilidades.
Cada camada resolve um único problema.

Gemini Notebook

A Camada de Dados e o Papel do ORM

- **PostgreSQL:** Persistência relacional pura (Tabelas, Registros, Relacionamentos).
- **O Problema:** Escrever SQL cru espalhado pela aplicação gera código frágil e difícil de manter.
- **A Solução (ORM):** `Object-Relational Mapping`. Mapeia classes em Python diretamente para tabelas no banco. O ORM traduz operações em objetos para queries SQL automaticamente.



Gemini Notebook

O Coração do Backend e as Regras de Negócio

O backend não é apenas o banco de dados. É a única camada na qual você pode confiar.

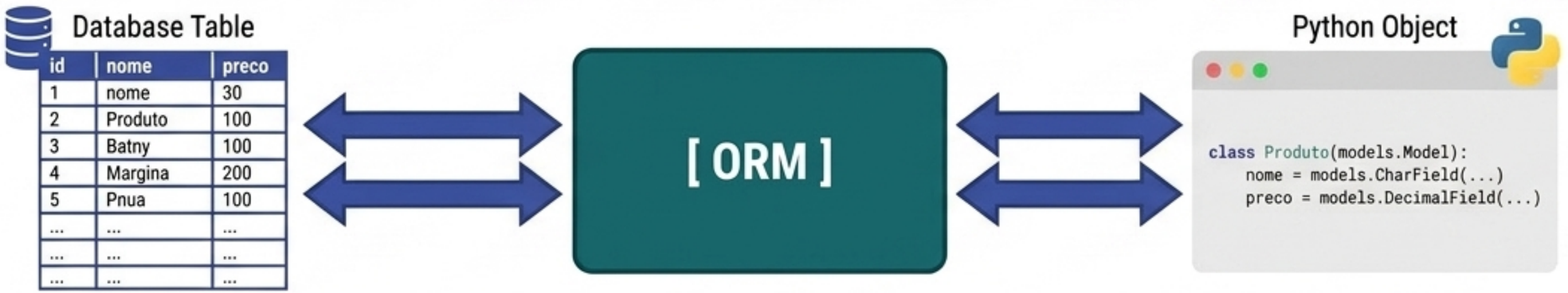


Nunca confie no Frontend para validação crítica. O servidor tem a palavra final.

Gemini Notebook

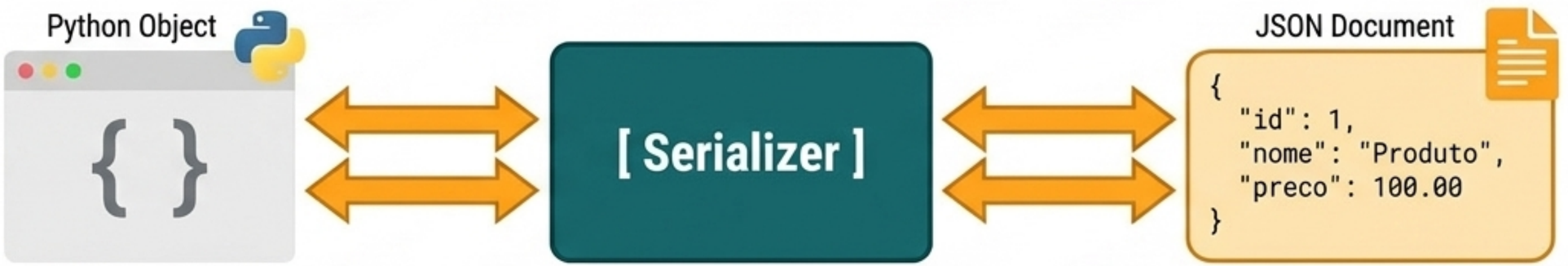
O Paradoxo da Tradução: ORM vs. Serializer

Raia 1:
O Acesso
aos Dados



ORM: Resolve o problema de falar com o banco de dados.
Traduz paradigmas (Relacional para Orientado a Objetos).

Raia 2:
A Comunicação
Externa



Serializer: Resolve o problema de falar com o mundo externo.
Transforma Objetos Python complexos em texto puro e universal (JSON) que trafega na web, e garante a validação dos dados que entram na API.

Gemini Notebook

A Ponte de Comunicação: Protocolo HTTP e API REST

REST define que interagimos com Recursos através de verbos padronizados.



Alert

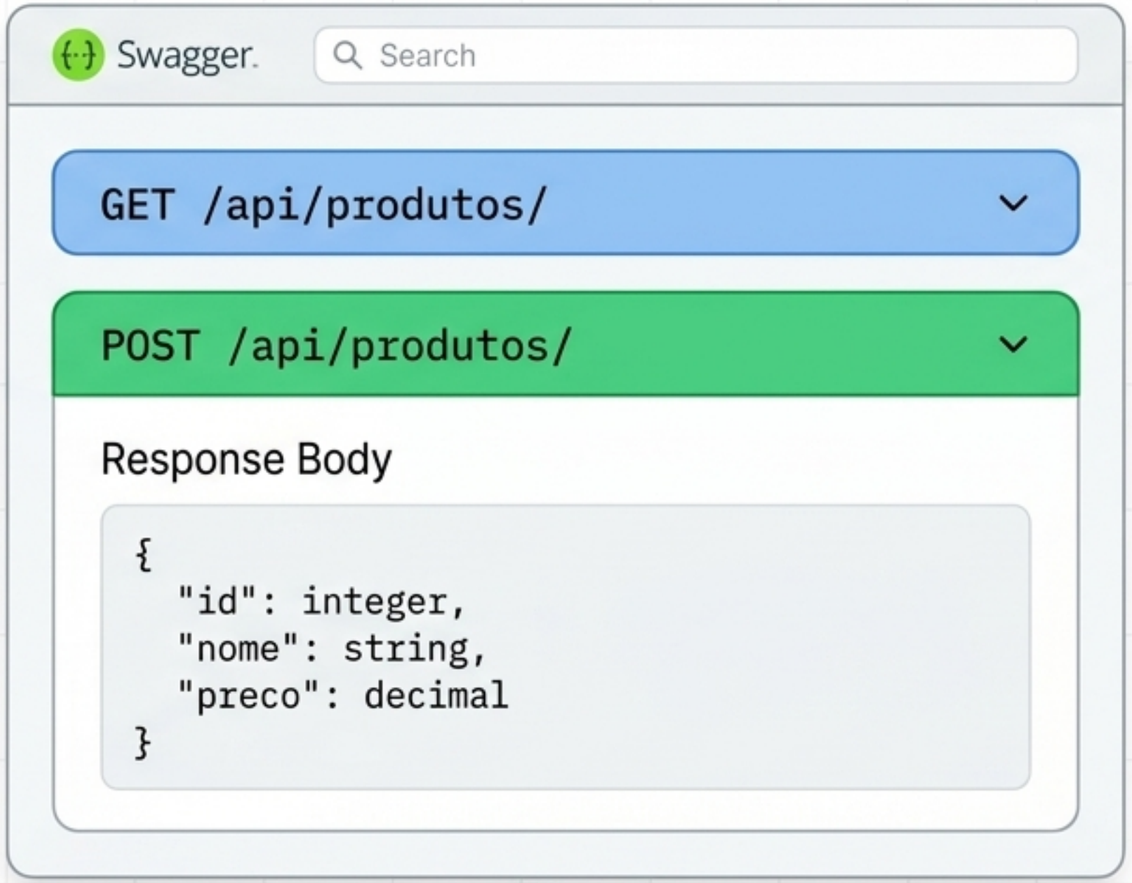
Status de Erro Comuns

- 400 Bad Request (Erro de validação)
- 401/403 (Problema de Acesso)
- 404 Not Found (Recurso inexistente)

Gemini Notebook

O Contrato de Paz: Swagger e OpenAPI

O Frontend não precisa adivinhar. A API documenta seu próprio funcionamento.

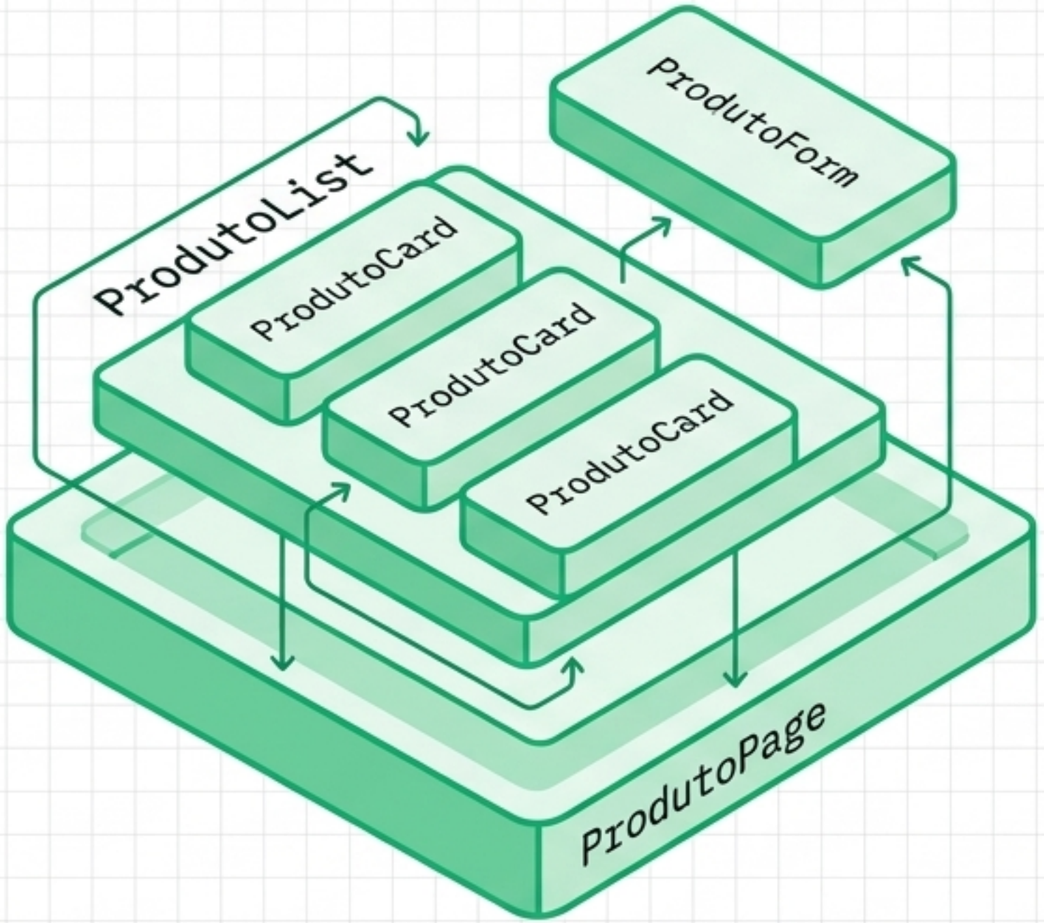


O que o Swagger responde?

- Quais endpoints estão disponíveis?
- Quais métodos (GET, POST) são aceitos?
- Quais dados (JSON) o Backend espera receber?
- Qual é o formato exato da resposta?

Gemini Notebook

O Frontend: Vue.js e Interfaces Reativas



Apresentar dados

Capturar interações humanas (cliques, digitação)

Manter o estado da tela (loaders, mensagens de erro)

Arquitetura de Componentes: Permite dividir uma interface complexa em partes menores, reutilizáveis e com responsabilidades únicas, evitando arquivos gigantescos de código.

Gemini Notebook

O Mensageiro: Axios e o Consumo de APIs

```
axios.get('/api/produtos/')
```

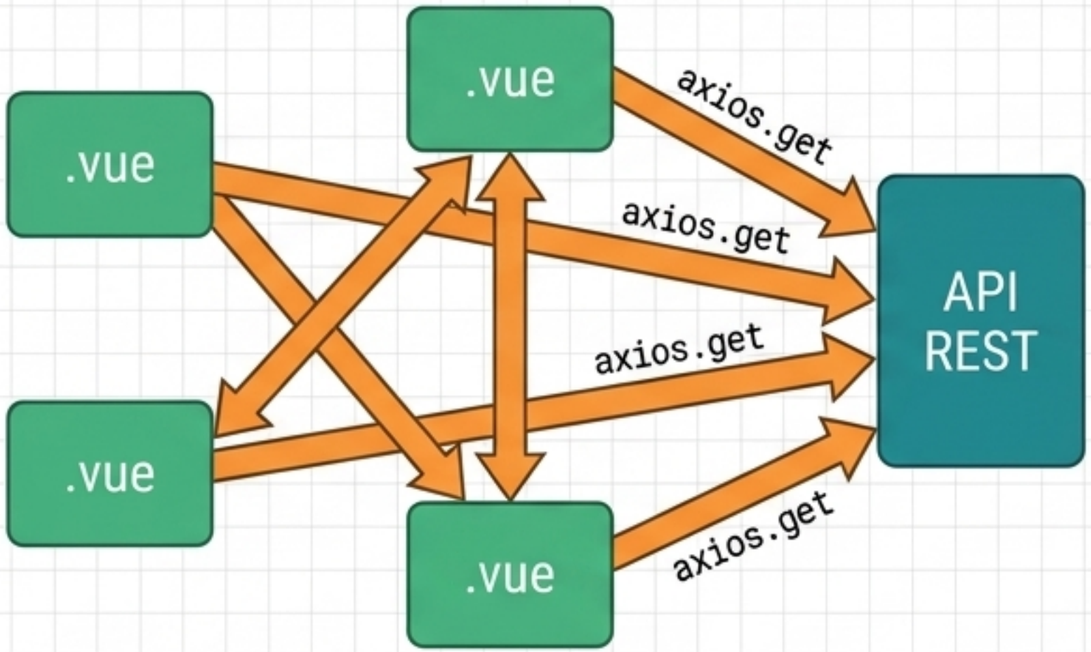


O Problema Resolvido: Transformar uma ação do usuário (clicar no botão 'Buscar') em uma requisição HTTP padronizada para a API e capturar a resposta para uso no JavaScript.

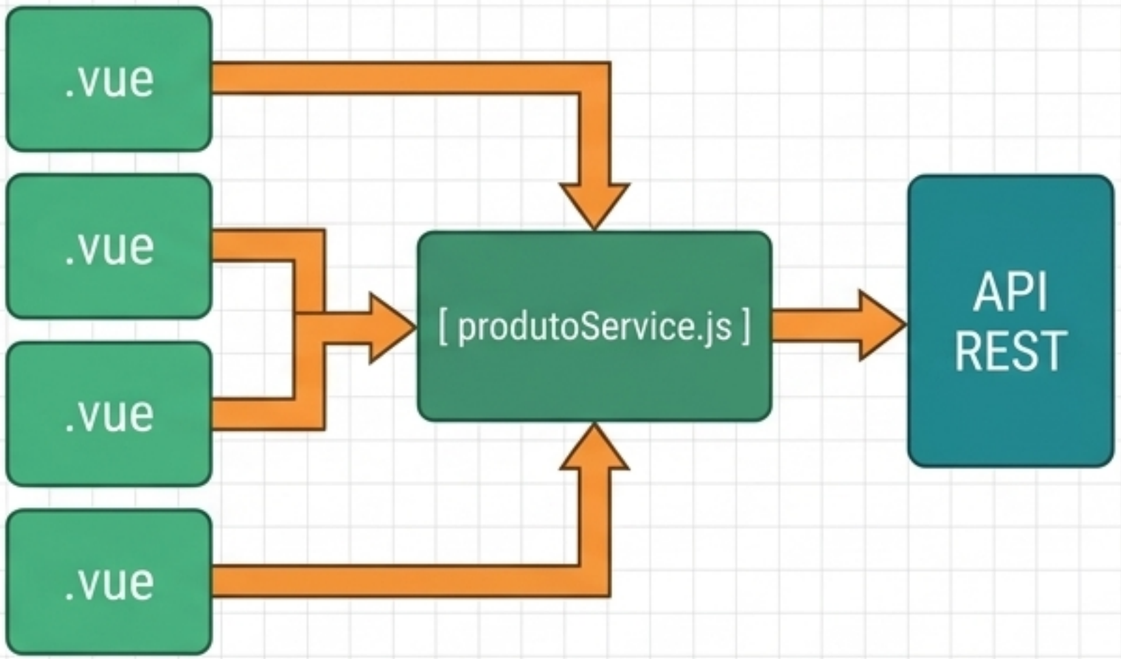
Gemini Notebook

Limpando a Casa: Arquitetura com Services

❌ **Antes** (Caótico) ❌



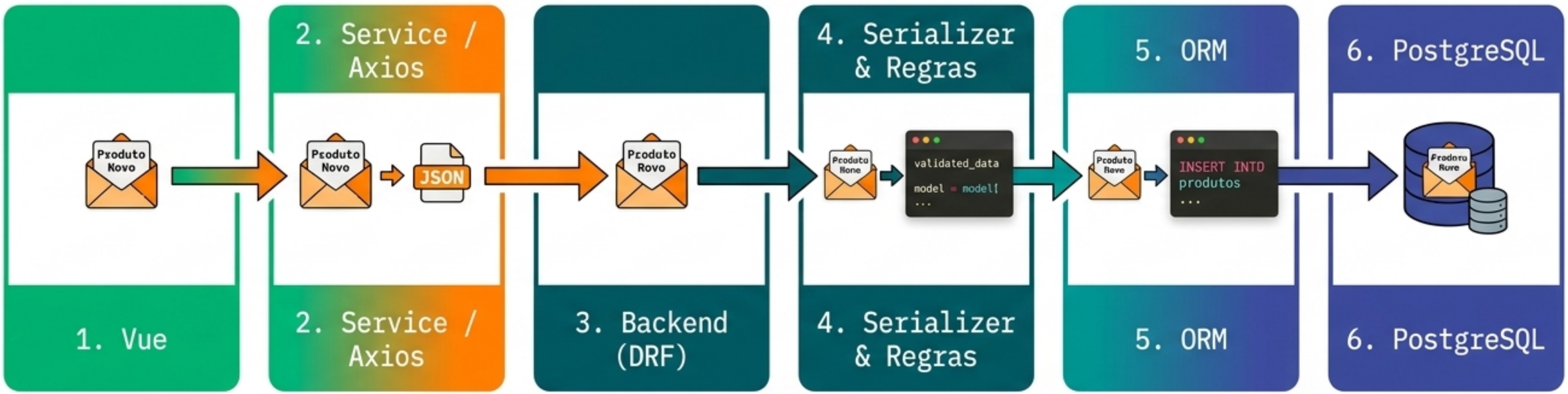
✅ **Depois** (Organizado) ✅



- **Evita duplicação**: A API é chamada de um só lugar.
- **Facilita manutenção**: Troca de URLs e tokens de autenticação centralizada.
- **Foco na Interface do Usuário**: O componente .vue deixa de se preocupar com infraestrutura.

Gemini Notebook

A Jornada da Requisição (O Caminho de Ida)



Usuário preenche formulário e clica em Salvar.

Empacota os dados em JSON e envia via HTTP POST.

Recebe a rota e aciona a View.

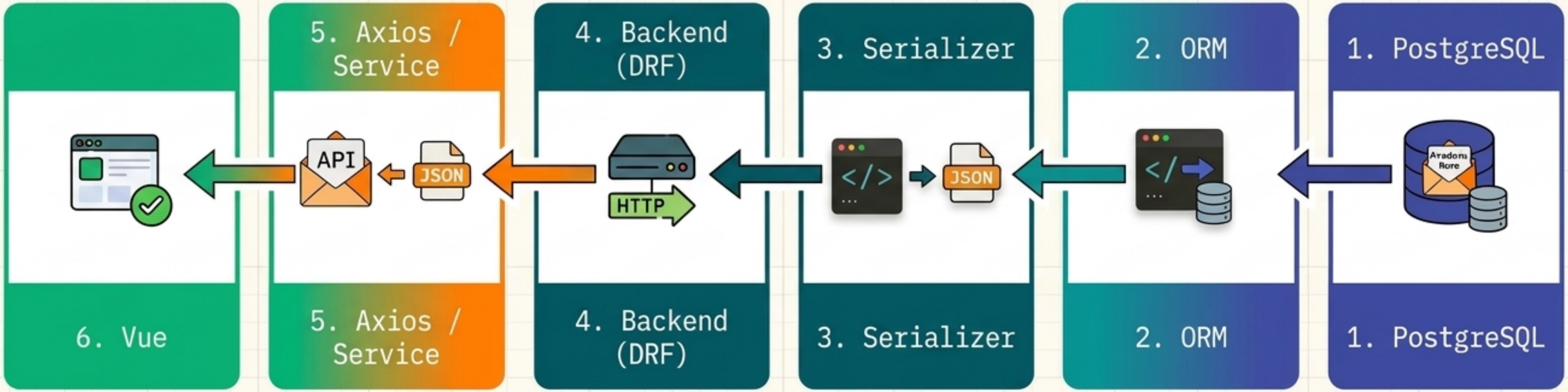
Converte JSON para dados Python. Valida as regras de negócio.

Traduz a instrução para linguagem de banco de dados (SQL).

Grava a nova linha no disco rígido.

Gemini Notebook

A Jornada da Resposta (O Caminho de Volta)



Confirma gravação e devolve o ID gerado.

Atualiza e devolve o Objeto Python.

Transforma o Objeto final em formato JSON puro.

Empacota o JSON em uma resposta HTTP 201 Created.

Recebe a resposta e a entrega descompactada ao Frontend.

Atualiza a tela (exibe uma notificação verde de sucesso ao usuário).

Gemini Notebook

O Mapa Mental Definitivo: Matriz de Responsabilidades

Tecnologia	Responsabilidade Exclusiva
PostgreSQL	Persistência bruta dos dados.
Django ORM	Mapeamento entre objetos da aplicação e banco de dados.
Backend (Django)	Estrutura do servidor e aplicação das Regras de Negócio.
DRF & Serializer	Construção da API, validação e tradução para JSON.
REST / HTTP	Protocolo universal de comunicação (A Ponte).
Swagger	Documentação do contrato de dados.
Axios & Services	Comunicação HTTP organizada no lado do cliente.
Vue.js	Apresentação da interface reativa via Componentes.

Gemini Notebook

Check-list de Compreensão

Você dominou a arquitetura se consegue explicar com suas palavras:

- ✅ A diferença de papéis entre o **Frontend (Interface)** e o **Backend** (Autoridade).
- ✅ Por que não podemos deixar regras de negócio críticas apenas na **UI (Vue.js)**.
- ✅ O **paradoxo da tradução**: a diferença exata entre **ORM** e **Serializer**.
- ✅ A função do **formato JSON** e do **protocolo HTTP** como o idioma universal da Web.
- ✅ Por que isolamos chamadas **Axios** em **Services** ao invés de deixá-las soltas nos componentes.

A tecnologia evolui, mas os padrões de arquitetura permanecem.

Gemini Notebook